越疆 Dobot

原子技能执行接口与双方联调手册

黑白专业交付版 | Implementation Interface Guide

项目	内容
适用对象	Dobot X-Trainer 双 Nova 原子技能执行网关及上游编排/编译系统
外部合同	RuntimeExecutionPackage；schema_version=2.1；graph_version=0.1
当前执行配置	dobot-dag/1 受限子集
实施基线	2026-10-11
证据边界	mock 验证接口与逻辑执行；live 才连接真实机械臂；command_trace 不等于物理抓放成功

使用声明

本文严格区分对方 Markdown 规范、当前已实现合同、历史设计稿和未实现能力。任何未在“当前支持范围”中列出的节点或字段，不得在联调时假定为可执行。

1. 文档结论与版本定位

本接口是依据对方《设备能力、原子技能库与动态执行图接口规范》1.0-draft
设计并实现的设备侧执行网关。对方规范定义了角色边界、目录版本、技能调用、执行图和事件合同；本实现将其中与 Dobot 双臂定点抓放相关的部分落地为 dobot-dag/1 受限子集。

1.1 四层关系

层次	作用	当前状态
对方 Markdown 规范	定义跨团队稳定合同、数据字段、角色边界和扩展方向	外部依据；版本为 1.0-draft
本地对照分析	识别字段/接口差异，确定可实现子集和缺口	已完成；不是新的外部合同
当前 HTTP 网关	接收运行包、校验、占用设备、执行技能、保存状态和事件	已实现 mock/fake/loopback 验证
内部 AtomicGraph	把 pick/place 展开成移动、夹爪、反馈检查等底层动作	内部实现；不要求对方直接发送

1.2 不得混用的历史格式

- skill-dag/1.0 是早期接口设计草案，不是当前正式外部提交格式。
- AtomicExecutionGraph schema_version=2.0 是内部执行图层，不是对方当前应提交的 RuntimeExecutionPackage。
- 当前外部包必须使用 schema_version=2.1、graph_version=0.1，并携带 catalog_revision。

结论：对方提交 RuntimeExecutionPackage；我方将其校验并确定性展开为内部原子动作。

2. 双方职责与接口方向

原子技能的“原子性”属于编排层：技能可以独立下发、观测、完成、取消或失败。它不等于设备驱动层的最小指令。对方不应直接发送关节角、任意坐标、开爪/闭爪等底层控制命令。

责任方	负责内容	不负责/不应做
对方编译/编排系统	工艺语义、约束推理、技能/设备选择、RuntimeExecutionPackage 生成、失败后的恢复决策	不直接拼接 Dobot 底层运动序列；不假定未登记节点可执行
Dobot 执行网关	能力目录、技能版本、图结构和场景校验；资源/设备锁；执行已登记 pick/place；状态、事件和错误返回	不自行扩大重试权限；不把未知条件静默映射；不替代急停或安全 PLC
现场操作员	确认机械臂静止、初始场景、急停可及、报警清除和夹爪/物体状态	不把 HTTP accepted 或 command_trace 当成物理成功

2.1 接口方向

方向	接口	说明
对方 → 我方	POST /device-gateway/v1/execution-plans/validate	运行包预检和资源预留，返回 validation_token
对方 → 我方	POST /device-gateway/v1/executions	携带 validation_token 正式执行
对方 → 我方	POST /v1/execute-graph	单次推送完整 graph，直接受理执行
我方 → 对方（当前代码可选配置）	GET /v1/integrations/execution-plans/{run_id}	网关按 run_id 从上游拉取运行包；需配置 compiler-url
我方 → 对方（当前代码可选配置）	目录/设备状态/执行事件回传路径	按上游 URL 配置；未配置时本地持久化，不声称已送达

3. 当前实现合同（必须以此联调）

当前网关实现的是对方规范的受限子集。预检阶段遇到不支持的结构必须拒绝，不能删除字段、静默转换或按 JSON 顺序猜测执行。

类别	当前支持
版本	schema_version=2.1；graph_version=0.1；profile=dobot-dag/1
节点	action、wait；节点初始 status 必须为 pending
技能	embodied.pick@1.0.0、embodied.place@1.0.0
设备	dobot-nova-left 绑定物体 A；dobot-nova-right 绑定物体 B
参数	pick: object_id + source_ref；place: object_id + destination_ref；只接受登记的 object/target 引用
边	仅 success；多条入边为 AND 依赖；entry_node_ids 必须等于全部依赖根节点
等待	duration_s 为 0–60 秒；不得超过 timeout_s；wait 不带 skill_invocation
超时	timeout_s 为 0.01–900 秒；默认 180 秒
并发	一个 rig 同时一个执行；mock 可跨臂并行；live 当前对共享工作区保守串行
幂等	HTTP 预检/提交需 Idempotency-Key 或 request_id；每个 action 还需唯一 skill_invocation.idempotency_key
场景	固定示教场景；不接受任意坐标；fresh-run 必须最终释放物体

3.1 当前明确不支持

不支持项	处理方式
condition、observe、parallel、human_approval、recovery 节点	预检拒绝；不得静默转换为 action/wait
failure/timeout/resource/cancel 等非 success 边	预检拒绝；当前只接受 success
断点续跑、resume_from_node_id、自动恢复图、本地自动重试	预检拒绝或失败后要求上游重新感知并提交新 fresh graph
任意关节角、任意 TCP 坐标、厂商底层命令	不属于当前外部技能合同

4. HTTP 控制面

方法	路径	用途	成功结果
GET	/health	检查模式、目录版本、活跃执行和隔离状态	status=ok
GET	/v1/catalog	获取设备、技能、绑定、执行 profile 和 catalog_revision	供对方编译锁定
POST	/v1/execute-graph	单次推送完整 graph	HTTP 202；返回 execution_id 等
POST	/device-gateway/v1/execution-plans/validate	两阶段第一步：预检/预留	HTTP 200；返回 validation_token
POST	/device-gateway/v1/executions	两阶段第二步：正式执行	HTTP 202；返回 execution_id
GET	/v1/executions/{id}	查询执行总状态、节点状态、错误和验证等级	HTTP 200
GET	/v1/executions/{id}/events?after=0&limit=100	分页查询事件；limit 最大 1000	HTTP 200
POST	/v1/executions/{id}/cancel	请求取消并进入停止收尾	HTTP 200；不等于已停止
GET	/v1/delivery-status	查询上游事件投递队列	HTTP 200

4.1 HTTP 约束

- 若服务端配置 DOBOT_GATEWAY_TOKEN，所有请求必须带 Authorization: Bearer <token>。
- POST 必须使用 Content-Type: application/json；不接受 chunked；单请求体最大 2 MiB。
- 重复使用相同幂等键和相同原始包不得重复物理动作；相同键换包返回冲突。
- 目录版本过期返回 STALE_CATALOG_REVISION；不得由网关静默替换 catalog_revision。
- live 失败或停止状态未知时会保持隔离；不能通过重复提交绕过。

5. 运行包 JSON 合同

HTTP 单次提交的外层是 {"graph": RuntimeExecutionPackage}；两阶段预检/提交的外层是 {"package": RuntimeExecutionPackage, ...}。以下 JSON 是 RuntimeExecutionPackage 本体。

字段	要求
schema_version / graph_version	固定为 2.1 / 0.1
package_id	稳定运行包标识；若出现 graph_id，必须等于 package_id
run_id / experiment_id	调用方运行和实验血缘；每次任务应可追溯
catalog_revision	必须来自当前 GET /v1/catalog；过期即拒绝
entry_node_ids	必须准确等于没有前驱依赖的根节点集合
nodes[].status	新提交必须为 pending；不支持伪造完成节点续跑
action.skill_invocation	skill_id、skill_version、device_id、idempotency_key、parameters
wait	只含 duration_s/timeout_s 等等待字段，不含 skill_invocation
edges	每条必须恰好含 source、target、edge_type；edge_type 当前只能是 success

5.1 单臂 A → 目标 A 示例

```
{
 "schema_version": "2.1",
 "graph_version": "0.1",
 "package_id": "runtime-single-001",
 "run_id": "single-001",
 "experiment_id": "experiment-single-001",
 "recipe_id": "dobot-single-pick-place",
 "version": "1.0",
 "catalog_revision": "REPLACE_WITH_CURRENT_CATALOG_REVISION",
 "entry_node_ids": ["pick-A"],
 "nodes": [
  {"node_id":"pick-A","node_type":"action","status":"pending","timeout_s":180,
   "skill_invocation":{"skill_id":"embodied.pick","skill_version":"1.0.0",
    "device_id":"dobot-nova-left","idempotency_key":"single-001:pick-A",
    "parameters":{"object_id":"A","source_ref":"object:A"}}},
  {"node_id":"place-A","node_type":"action","status":"pending","timeout_s":180,
   "skill_invocation":{"skill_id":"embodied.place","skill_version":"1.0.0",
    "device_id":"dobot-nova-left","idempotency_key":"single-001:place-A",
    "parameters":{"object_id":"A","destination_ref":"target:target_A"}}}
 ],
 "edges": [
  {"source":"pick-A","target":"place-A","edge_type":"success"}
 ]
}
```

6. 双臂执行图示例

该图对应：左臂抓 A 并保持 5 秒，右臂抓 B 并放到目标 B；随后左臂放 A 到目标 A；最后右臂从目标 B 抓 B 并放回 B 初始位置。


```
{
 "schema_version": "2.1",
 "graph_version": "0.1",
 "package_id": "runtime-bimanual-001",
 "run_id": "bimanual-001",
 "experiment_id": "experiment-bimanual-001",
 "recipe_id": "dobot-hold-return",
 "version": "1.0",
 "catalog_revision": "REPLACE_WITH_CURRENT_CATALOG_REVISION",
 "entry_node_ids": ["pick-A", "pick-B"],
 "nodes": [
  {"node_id":"pick-A","node_type":"action","status":"pending","timeout_s":180,
   "skill_invocation":{"skill_id":"embodied.pick","skill_version":"1.0.0",
    "device_id":"dobot-nova-left","idempotency_key":"bimanual-001:pick-A",
    "parameters":{"object_id":"A","source_ref":"object:A"}}},
  {"node_id":"pick-B","node_type":"action","status":"pending","timeout_s":180,
   "skill_invocation":{"skill_id":"embodied.pick","skill_version":"1.0.0",
    "device_id":"dobot-nova-right","idempotency_key":"bimanual-001:pick-B",
    "parameters":{"object_id":"B","source_ref":"object:B"}}},
  {"node_id":"hold-A","node_type":"wait","status":"pending","duration_s":5,"timeout_s":60},
  {"node_id":"place-A","node_type":"action","status":"pending","timeout_s":180,
   "skill_invocation":{"skill_id":"embodied.place","skill_version":"1.0.0",
    "device_id":"dobot-nova-left","idempotency_key":"bimanual-001:place-A",
    "parameters":{"object_id":"A","destination_ref":"target:target_A"}}},
  {"node_id":"place-B","node_type":"action","status":"pending","timeout_s":180,
   "skill_invocation":{"skill_id":"embodied.place","skill_version":"1.0.0",
    "device_id":"dobot-nova-right","idempotency_key":"bimanual-001:place-B",
    "parameters":{"object_id":"B","destination_ref":"target:target_B"}}},
  {"node_id":"repick-B","node_type":"action","status":"pending","timeout_s":180,
   "skill_invocation":{"skill_id":"embodied.pick","skill_version":"1.0.0",
    "device_id":"dobot-nova-right","idempotency_key":"bimanual-001:repick-B",
    "parameters":{"object_id":"B","source_ref":"target:target_B"}}},
  {"node_id":"return-B","node_type":"action","status":"pending","timeout_s":180,
   "skill_invocation":{"skill_id":"embodied.place","skill_version":"1.0.0",
    "device_id":"dobot-nova-right","idempotency_key":"bimanual-001:return-B",
    "parameters":{"object_id":"B","destination_ref":"object:B"}}}
 ],
 "edges": [
  {"source":"pick-A","target":"hold-A","edge_type":"success"},
  {"source":"hold-A","target":"place-A","edge_type":"success"},
  {"source":"pick-B","target":"place-B","edge_type":"success"},
  {"source":"place-A","target":"repick-B","edge_type":"success"},
  {"source":"place-B","target":"repick-B","edge_type":"success"},
  {"source":"repick-B","target":"return-B","edge_type":"success"}
 ]
}
```


注意：示例中的 catalog_revision、run_id、package_id、experiment_id
和所有幂等键仅为占位。对方每次实际提交必须生成新值，并先读取当前 catalog_revision。

7. Kunyu 侧：本地 Mock 联调步骤

Mock 的含义是：网关仍接收真实 HTTP JSON，但执行器使用
MockAdapter，不连接机械臂、不发送运动或夹爪命令。demo_client 是“假对方”；MockAdapter 是“假机器人”。

7.1 启动 Mock 网关（终端一）

```
$PY = 'C:\software\anaconda3\envs\dobot-xtrainer\python.exe'
$ROOT = '\\wsl.localhost\Ubuntu20.04\home\yky\.codex\work\dobot_xtrainer_teleop_fix'
$MOCK = 'C:\DobotRuntime\gateway-data\mock-001'
$env:DOBOT_GATEWAY_TOKEN = 'local-mock-token'
$env:PYTHONPATH = $ROOT

& $PY -X utf8 -m task_skills.run_gateway --host 127.0.0.1 --port 8080 --mode mock --data-dir $MOCK
```

7.2 模拟对方提交（终端二）

```
$PY = 'C:\software\anaconda3\envs\dobot-xtrainer\python.exe'
$ROOT = '\\wsl.localhost\Ubuntu20.04\home\yky\.codex\work\dobot_xtrainer_teleop_fix'
$env:DOBOT_GATEWAY_TOKEN = 'local-mock-token'
$env:PYTHONPATH = $ROOT

& $PY -X utf8 -m task_skills.gateway.demo_client --url http://127.0.0.1:8080 --two-phase
```

- 预期：demo_client 先 GET /v1/catalog，再进行 validate → executions → status/events。
- 预期结果：返回 execution_id，最终 status=succeeded；结果带 hardware_connected=false 或 mock 证据。
- 每次重新测试建议使用新 data-dir；同一 mock 目录会保存逻辑物体状态和幂等历史。

8. Kunyu 侧：真机网关部署与安全门

以下步骤只在 mock 合同联调通过、现场两臂状态已确认后执行。live 网关是 Windows 硬件环境入口，必须显式 --mode live --allow-live 并配置令牌。

8.1 现场先决条件

- 两台 Nova 静止、无报警、控制网络可达；急停可触达；运动包络内无人员和障碍物。
- A/B 已人工确认处于 scene_config.json 对应初始示教位置；目标位置无占用；夹爪状态已确认。
- 停止旧 run_task、run_control.py、launch_nodes.py、旧 ZMQ/gateway 进程；同一 rig 只能有一个控制所有者。
- 若存在 stale device-owner 或 scene-invalid，不能直接删除后重跑；必须先确认 PID 已退出并完成人工场景复位。

8.2 记录现场复位并启动 live 网关

```
$PY = 'C:\software\anaconda3\envs\dobot-xtrainer\python.exe'
$ROOT = '\\wsl.localhost\Ubuntu20.04\home\yky\.codex\work\dobot_xtrainer_teleop_fix'
$DATA = 'C:\DobotRuntime\gateway-data\live'
$LOCK = 'C:\DobotRuntime\device-owner'
$env:DOBOT_DEVICE_LOCK_PATH = $LOCK
$env:DOBOT_GATEWAY_TOKEN = '<现场注入，不写入脚本或日志>'
$env:PYTHONPATH = $ROOT

& $PY -X utf8 -m task_skills.gateway.operator --mode live --data-dir $DATA --lease-path $LOCK --confirm RESET_VERIFIED

& $PY -X utf8 -m task_skills.run_gateway --host 0.0.0.0 --port 8080 --mode live --allow-live --allow-insecure-http --data-dir $DATA
```

传输安全：上面的 --allow-insecure-http 只允许局域网临时实验；生产或跨网络联调必须改用 --tls-cert 与 --tls-key，并使用令牌。当前实现不是生产级 mTLS/OAuth2 实现。

9. 对方侧：编译、预检、提交和轮询

对方应把本节作为接入顺序。对方不需要调用中文 run_task，也不需要知道 Dobot 内部 43 节点展开。

9.1 获取目录并编译

```
GET /v1/catalog
Authorization: Bearer <token>
```

使用返回的 catalog_revision 编译 RuntimeExecutionPackage。
必须校验 device_id、skill_id、skill_version、参数和固定对象/目标引用。

9.2 两阶段协议（推荐）

```
POST /device-gateway/v1/execution-plans/validate
Authorization: Bearer <token>
Idempotency-Key: preflight-<run_id>
Content-Type: application/json

{"package": <RuntimeExecutionPackage>, "reserve_resources": true}
```

成功后保存 validation_token；令牌过期、目录变化或资源不可用时重新编译/预检。

```
POST /device-gateway/v1/executions
Authorization: Bearer <token>
Idempotency-Key: dispatch-<run_id>
Content-Type: application/json

{"package": <同一份RuntimeExecutionPackage>, "validation_token": "<token>"}
```

9.3 单次推送协议

```
POST /v1/execute-graph
Authorization: Bearer <token>
Idempotency-Key: dispatch-<run_id>
Content-Type: application/json

{"graph": <RuntimeExecutionPackage>}
```

9.4 查询、事件和取消

```
GET  /v1/executions/{execution_id}
GET  /v1/executions/{execution_id}/events?after=0&limit=100
POST /v1/executions/{execution_id}/cancel
```

取消请求被接受不等于机械臂已停止；必须继续查询到 cancelled、failed 或 interrupted。

10. 双方联调验收清单

阶段	Kunyu / Dobot 侧	对方编排侧	通过条件
合同	提供当前 catalog 与 profile	读取并锁定 catalog_revision	双方使用同一 revision
Mock	启动 mock 网关，保存 execution/events	发送单臂、双臂和非法图	合法图成功；非法图稳定拒绝
幂等	保存请求和节点幂等记录	重复发送相同键和相同包	不产生重复执行；换包返回冲突
依赖	校验根节点、环、同臂冲突和 AND 依赖	提交双臂并行/汇合图	顺序与边语义一致
错误	返回稳定 code、node_id、details	按 retryable 和 on_fail 做外部决策	不由网关自行扩大重试
事件	提供状态和分页事件	按 execution_id/sequence 消费	可去重、发现断序
Live 前	确认两臂、物体、夹爪、急停和锁	提交 fresh graph，不假设 resume	现场 operator gate 通过
Live	低速、人工监护、必要时取消	继续轮询并记录证据等级	先 command_trace，再单独确认物理结果

10.1 错误处理原则

错误	对方应做什么
STALE_CATALOG_REVISION	重新 GET /v1/catalog，更新 catalog_revision，重新编译；不得静默替换
UNSUPPORTED_NODE_TYPE / EDGE_TYPE / CONDITION	修改图到当前支持子集；不得删字段后继续执行
IDEMPOTENCY_CONFLICT	停止重试，核对 run_id、请求键和节点键是否复用
RESOURCE_CONFLICT / ROBOT_BUSY	查询当前执行和设备状态；不得并发抢占同一 rig
STOP_UNCONFIRMED / interrupted	现场确认机械臂/夹爪静止和物体状态，复位后提交新 fresh graph
CONTROLLER_ALARM / MOTION_TIMEOUT	保留证据并走外部恢复决策；当前网关不自动重抓

11. 证据等级、限制和发布门

本系统必须分层报告结果，不能把接口成功、文件落盘或 command_trace 直接写成物理任务成功。

证据层	能证明什么	不能证明什么
HTTP 202 / execution_id	网关接受了请求并创建执行会话	不能证明节点完成或物体抓取
mock succeeded	合同、校验、逻辑调度和事件链路	不能证明硬件连接、运动、碰撞或抓放
live command_trace	控制器指令、反馈等待和技能内部步骤记录	不能单独证明物体实际抓稳、放准或工艺质量
现场/独立传感器确认	可补充物体状态和物理结果证据	仍需明确判据、时间和对象对应关系

11.1 当前不能宣称的内容

- 不能宣称完整兼容对方 Markdown 规范中所有动态节点、条件、恢复和事件 ACK 能力。
- 不能把 mock 的双臂逻辑并行当成 live 真机双臂物理并行成功。
- 不能把一次 command_trace 或单次人工目视写成成功率、泛化能力或工艺质量。
- 不能在 live 失败后直接删除锁、跳过现场复位或重复提交旧图。

11.2 发布门

- 双方先以当前受限 JSON 合同完成 mock 合同测试。
- 拿到对方真实运行包和最终 Schema 后，先做两端 mock 合同联调。
- 现场 live 前完成初始场景确认、唯一锁路径确认、readiness 和低速验证。
- 若对方要求 condition/observe/recovery/resume 等能力，应先补合同和实现，再扩大支持范围。

12. 可追溯来源与实施状态

本手册根据以下材料和当前代码生成。源材料中的“建议/草案”与当前代码中的“已实现”已分开标注。

材料	用途	状态
对方《设备能力、原子技能库与动态执行图接口规范》1.0-draft	外部合同依据：角色、目录、RuntimeExecutionPackage、两阶段协议和事件方向	对方草案；需双方最终确认
外部接口规范对照分析_20261005.md	识别 2.1 合同与本地实现差异；确定受限子集	已归档分析
implemented/接入说明.md	当前网关接入、命令、支持范围、live 安全流程	当前实施说明
task_skills/gateway/server.py / contracts.py	当前实际路由和字段校验依据	代码事实；应优先于旧设计稿

最终状态格式：当前阶段：mock/合同联调；证明：HTTP、Schema、幂等、逻辑调度和事件；不证明：真实物理抓放和工艺质量；阻塞：对方最终 Schema/真实运行包/ACK 合同与现场 live gate；下一门：双方 mock 合同测试通过后再做低速 live。

交付说明

本 PDF 是基于当前代码和已知外部草案的实施联调手册，不替代双方签署的正式 JSON Schema、网络安全协议、现场安全规程或急停/PLC 设计。